

Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Understanding React's Lifting State Up with a Simple Example



Devendra Dhote

Follow

2 min read · Jul 3, 2026





LIFTING STATE UP IN REACT

Share State.
Stay in Sync.

```
function App() {  
  const [users, setUsers] = useState([]);  
  return (  
    <>  
      <Form setUsers={setUsers} />  
      <UserCards users={users} />  
    </>  
  );  
}
```



When I first started learning React, I used to create state inside the component where it was needed.

It worked perfectly... until two components needed access to the same data.

That's where I discovered one of React's most important design patterns:
Lifting State Up.

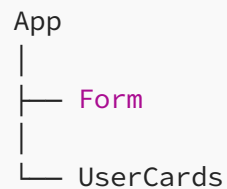
Let's understand it with a simple example.

. . .

The Scenario

Imagine we are building a small user management application.

We have two components:



- **Form** is responsible for collecting user information.
- **UserCards** is responsible for displaying all submitted users.

Initially, I created the state inside the `Form` component.

```
function Form() {  
  const [users, setUsers] = useState([]);
```

```
  // ...  
}
```

Everything looked fine.

Until I realised something...

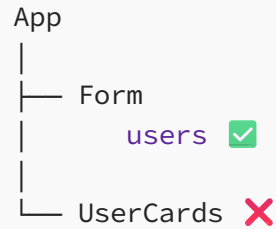
How will `UserCards` access the `users` state?

It can't.

Because the state belongs only to the `Form` component.

. . .

The Problem



Since `UserCards` is a sibling of `Form`, it has no direct access to the state stored inside `Form`.

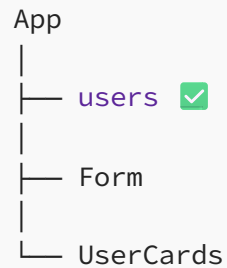


This is where many React beginners get stuck.

. . .

The Solution: Lift the State Up

Instead of storing the state inside `Form`, move it to their nearest common parent.



Now the parent component owns the state and simply shares it with its children using props.

```
function App() {  
  const [users, setUsers] = useState([]);
```

```
  return (  
    <>  
      <Form setUsers={setUsers} />  
      <UserCards users={users} />  
    </>  
  );  
}
```

Now the flow becomes much cleaner.

Form

↓

setUsers()

↓

App State Updates

↓

App Re-renders

↓

Updated Props

↓

UserCards Re-renders

• • •

Why Is This Better?

Keeping shared state in the nearest common parent gives us:

- A single source of truth
- Better data flow
- Easier debugging
- Better component communication
- Cleaner and more maintainable code

Instead of multiple components maintaining separate copies of the same data, they all rely on one shared state.

. . .

Final Thoughts

One of the biggest lessons I learned while working with React is this:

If multiple components need the same data, don't duplicate the state — lift it up to their nearest common parent.

This simple design pattern makes your applications easier to understand, easier to maintain, and aligns perfectly with React's one-way data flow philosophy.

Sometimes, writing better React isn't about adding more code — it's about placing your state in the right place.

Happy coding ❤️

React ✓

JavaScript ✓

Web Development ✓



Written by Devendra Dhote

284 followers · 8 following

Follow

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.



Placeholder text for a bio or header.

Placeholder text for a list of items or a detailed bio.